<u>EXPERIMENT 2</u>
THIVYAA P
240701570

I implemented a To-Do List application across three different interface paradigms — CLI, GUI, and VUI — using Python, to study how the choice of interaction modality influences the overall user experience, ease of learning, and mental effort required.

Familiar and Unfamiliar Interaction in UI/UX Design

Familiar Interaction refers to design patterns and interaction styles that users have previously encountered in everyday applications. These are easy to pick up without instruction and feel natural from the first use.

Eg: Text input boxes and submit buttons (CLI/GUI), saying ‘exit’ or ‘stop’ to end a session (VUI).

Unfamiliar Interaction refers to less conventional design choices that users may not immediately understand. While they can enhance the experience once learned, they introduce an initial period of adaptation.

Eg: Pressing Enter to add a task instead of clicking a button (GUI), speaking a task number as a word such as ‘three’ instead of typing 3 (VUI).

FRAME 1:
CLI (Command-Line Interface):

The CLI version of the To-Do List runs entirely inside a terminal window. Users interact by reading a printed menu and typing a number to select an action. No graphical elements or mouse input are involved. The application relies solely on Python’s built-in input and print functions.

<u>Familiar Navigation Elements</u>

- A simple numbered menu (1 to 4) presented at the start of each cycle, allowing users to select an action by typing a digit — a pattern common in console-based tools.

- Step-by-step prompting: the application asks for input only after a choice is made, keeping the flow predictable and easy to follow.
- Friendly error messages such as 'Task cannot be empty' and 'Invalid task number' help users correct mistakes without confusion.

<u>Unfamiliar Navigation Elements</u>

- Users cannot see the full task list at all times; they must explicitly select option 2 to view tasks, which feels passive compared to visual interfaces.
- The lack of any colour, icon, or formatting makes it harder to scan and distinguish between different parts of the output at a glance.

FRAME 2:
GUI (Graphical User Interface):

The GUI version is built using Python's Tkinter library and opens as a standalone desktop window. It presents all controls simultaneously — an input field, action buttons, a scrollable task list, and a live status bar — allowing users to manage tasks without memorising any commands.

<u>Familiar Navigation Elements</u>

- A clearly labelled text entry box paired with an 'Add' button follows the standard form-input pattern found in most desktop and web applications.
- Tasks are displayed in a scrollable numbered list that updates immediately after each action, giving users constant visibility of their data.
- A confirmation dialog appears before clearing all tasks, following the widely understood 'Are you sure?' pattern to prevent accidental data loss.

<u>Unfamiliar Navigation Elements</u>

- Pressing the Enter key while typing in the input field automatically adds the task — a keyboard shortcut that mouse-first users may not expect or discover.
- The status label changes its text colour depending on the type of action taken (green for additions, red for removals), which is a visual feedback convention not all users recognise immediately.

FRAME 3:
VUI (Voice User Interface):

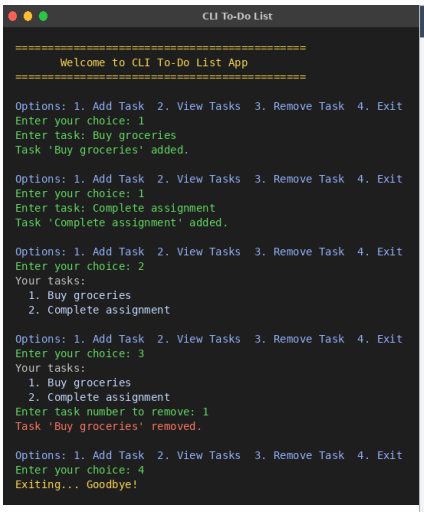
The VUI version operates entirely through voice. It uses the SpeechRecognition library to listen to spoken commands via the microphone, and the pyttsx3 library to respond through text-to-speech audio. There is no screen to look at — every action, prompt, and response happens through sound alone.

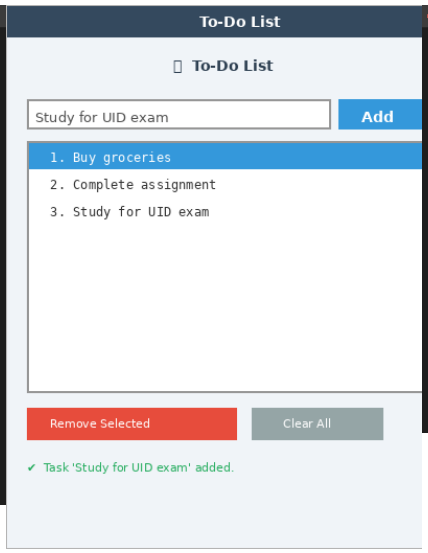
Familiar Navigation Elements

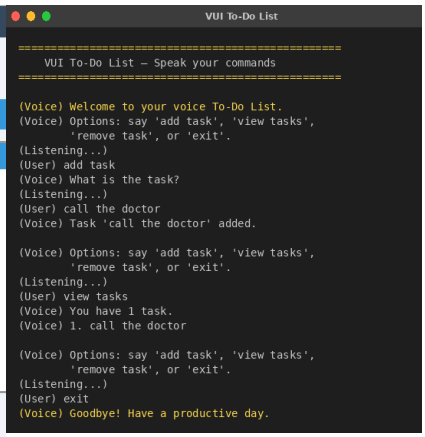
- Commands like 'add task', 'view tasks', 'remove task', and 'exit' closely match how people speak to everyday voice assistants such as Alexa or Google Assistant.
- After every action, the system speaks a confirmation aloud — for example, 'Task added' or 'Task removed' — which is the expected feedback pattern in voice-driven systems.
- The menu options are re-read at the beginning of every turn, ensuring the user always knows what they can do next without needing to remember.

Unfamiliar Navigation Elements

- To remove a task, users must speak its number as a word (e.g. 'two') rather than saying a digit, which is an uncommon requirement that may cause confusion on first use.
- Because there is no screen, users cannot review what was said or what the system captured; if recognition fails silently, there is no visible indication of the error.
- The system requires an active internet connection for the Google Speech Recognition API, which is an invisible dependency that users may not be aware of until it fails.







Result

All three versions of the To-Do List application were successfully built and tested using Python. The CLI application ran in the terminal with no dependencies. The GUI application launched as a desktop window using Tkinter, providing a visual and interactive experience. The VUI application responded to spoken commands through the microphone and gave audio feedback using text-to-speech. The experiment demonstrated that the same core functionality can feel very different depending on the interface used, and that each modality carries its own advantages, limitations, and learning curve for different types of users.